

Student Project Guide — RAG-Powered Document Assistant

Work mode	Individual assignment
Deadline	4 days from the date the assignment is issued
Submission	Backend source code, database schema, setup instructions, and API documentation

Important: Each student must design and implement the project independently. Shared implementations or copied submissions are not permitted.

Goal: Build a complete AI product, from raw documents to a deployed, working RAG (Retrieval-Augmented Generation) web application, and publish it on GitHub. Time limit: 6 days. Teams: individual or 2–3 students.

You will:

- Choose any domain and collect a set of source documents (Extended Track: also an image dataset)
- Build a notebook that cleans and chunks the data, generates embeddings, stores them in a vector database, and builds and evaluates a RAG pipeline using a local Ollama LLM
- Build a FastAPI backend that serves the RAG pipeline
- Build a frontend (Streamlit or Gradio) where a user asks a question and sees a grounded, cited answer
- Publish everything to GitHub with a professional README

Two tracks are available — see Phase 1 for details. Core Track builds a text-only RAG assistant; Extended Track adds a Computer Vision/YOLO component for teams who want an added multimodal challenge.

Phase 0 — Prerequisites & Environment Setup

Install and verify each of these before starting:

Tool	Minimum version	Check with
Python	3.10	<code>python --version</code>
Ollama	latest	<code>ollama --version</code>
Git	any recent	<code>git --version</code>
A GitHub account	—	https://github.com

Create your project folder and a virtual environment:

```
mkdir rag-assistant-project
cd rag-assistant-project
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # macOS / Linux
pip install jupyter pandas numpy chromadb sentence-transformers pypdf ollama python-dotenv
```

Phase 1 — Domain & Data Collection

Track Options:

- Core Track — a text-based RAG assistant (Chat with Documents/PDF)
- Extended Track — the Core Track pipeline enhanced with a Computer Vision/YOLO component (e.g. scanned pages, diagrams, product photos, or detection results feeding into the RAG context) for teams who want a multimodal challenge

Steps:

- Choose an open domain/topic (study notes, product manuals, legal or medical text, customer support FAQs, etc.) — any domain is fine as long as it produces a meaningful document collection
- Collect a set of source documents (PDFs/text) — enough to give the assistant something real to retrieve from
- **[Extended]** Extended Track: also collect a relevant image dataset (scanned pages, diagrams, product photos, etc.)
- Verify your data yourself — open a few files, check they are text-extractable (not scanned images needing OCR), and note anything messy you'll need to clean

Phase 2 — The Notebook: Build & Evaluate the RAG Pipeline

Create notebooks/rag_pipeline.ipynb. It must contain all of the following sections (use markdown headers so it reads like a report).

2.1 Load & Inspect

Write a short markdown cell answering: how many documents/pages? What formats? Which files failed to parse or need OCR?

2.2 Chunking Strategy

- Split documents into chunks (fixed-size with overlap, or a semantic/section-based strategy)
- Justify your chosen chunk size and overlap in a markdown cell

2.3 Embeddings & Vector Store

- Generate embeddings for each chunk
- Store embeddings in a vector database (e.g. Chroma or FAISS)
- Persist the vector store to disk so the backend can load it without rebuilding

2.4 Retrieval & Prompting

- Implement a retrieval function and test it against at least 10 sample questions
- Build the prompt template that combines the retrieved context with the user's question
- Add citation-style grounding — the answer should reference which chunk/document it came from

2.5 Vision Component

- **[Extended]** Run inference with, or fine-tune, a pretrained YOLO/CV model on the image dataset
- **[Extended]** Decide how detection/classification output feeds into the RAG prompt context

2.6 Evaluation

- Report results for at least 10 test questions: was the retrieved context relevant? was the answer grounded or hallucinated?
- Include a small results table (question / retrieved source / answer / correct or not)
- Write a short paragraph on the main failure cases you observed and how you mitigated them

2.7 Export

Save the persisted vector store (and any config such as chunk size, embedding model name) into a folder your backend will load directly — no rebuilding at request time.

Phase 3 — Backend (FastAPI)

Structure your backend as follows:

```
backend/
├── app/
│   ├── main.py           # FastAPI app, CORS, startup loading
│   ├── api/routes/query.py # GET /health, POST /query
│   ├── core/config.py    # Settings from .env
│   ├── schemas/query.py  # QueryRequest / QueryResponse
│   ├── services/
│   │   ├── retrieval.py  # Load vector store, retrieve chunks
│   │   └── generation.py # Call Ollama LLM, build answer
│   └── utils/logging_config.py
├── data/vector_store/    # copied from your notebook
├── tests/test_query.py
├── requirements.txt
├── .env.example
└── Dockerfile
```

Steps:

- `pip install fastapi "uvicorn[standard]" pydantic pydantic-settings ollama chromadb pytest httpx` and freeze into `requirements.txt`
- Define `QueryRequest {question: str}` and `QueryResponse {answer: str, sources: list[str]}`
- Implement `POST /query` (retrieve → build prompt → call LLM → return grounded answer) and `GET /health`
- Load the vector store and LLM connection once at startup (FastAPI lifespan), not on every request
- Add CORS middleware allowing your frontend's origin
- **[Extended]** Extended Track: add an endpoint (or an optional field on `/query`) for image-based input, and fuse detection output into the prompt
- Write at least 2 tests with `TestClient`: one happy path, one invalid input (expect 422)

Run and verify:

```
uvicorn app.main:app --reload
# open http://localhost:8000/docs and test /query from Swagger UI
```

Phase 4 — Frontend (Streamlit or Gradio)

Structure your frontend as follows:

```
frontend/
├── app.py                # main Streamlit/Gradio app
├── api_client.py         # wrapper for calling the backend API
├── .env                  # API_BASE_URL=http://localhost:8000
└── requirements.txt
```

Requirements:

- A chat-style interface: text input for questions, response area showing the answer and its cited sources
- Read the backend URL from an environment variable — never hard-code it
- **[Extended]** Extended Track: allow an image upload and display detection/classification results alongside the text answer
- Show a loading state while the request runs, and a friendly error message if the API fails
- Verify the full flow: backend on 8000, frontend running, ask a real question, see a grounded, cited answer

Phase 5 — Publish on GitHub

Create .gitignore before your first commit. It must exclude: .env/, __pycache__/, .env, *.log, the raw document corpus if large (explain in the README how to obtain it), and the vector store if large. Small models/artifacts may be committed if under 50 MB.

Initialise and commit:

```
git init
git add .
git commit -m "RAG assistant: notebook, FastAPI backend, frontend"
```

Create a public repository on GitHub, then:

```
git remote add origin https://github.com/<your-username>/rag-assistant-app.git
git branch -M main
git push -u origin main
```

Write a root README.md. It must include: overview, architecture diagram, tech stack, project structure, domain/data description, backend & frontend setup steps, environment variables table, API reference with a curl example, evaluation results (from Phase 2.6), and screenshots of the running app.

Verify like a stranger: clone your repo into a fresh folder and follow only your README. If any step fails, fix the README.

Final Presentation

- A live demo delivered in front of the instructors
- A recorded video walkthrough of the application

Deliverables Checklist

- ☐ notebooks/rag_pipeline.ipynb — runs top-to-bottom without errors, with chunking, embeddings, retrieval testing, and an evaluation table
- ☐ backend/ — FastAPI app with /health + /query, .env.example, pinned requirements.txt, passing pytest
- ☐ frontend/ — working chat interface, .env.example
- ☐ A persisted vector store, produced by the notebook and served by the backend
- ☐ Root README.md good enough for a stranger to run the whole project
- ☐ Public GitHub repository with a clean history (no .venv, no .env, no raw corpus dump)
- ☐ End-to-end demo works: question → API → retrieval → LLM → grounded answer on screen
- ☐ Extended Track only: working CV/YOLO component integrated into the pipeline
- ☐ Live demo delivered + recorded video submitted

Common Mistakes That Lose Points

- Committing .env, the raw document corpus, or the vector store when it's large
- Hard-coding http://localhost:8000 in frontend code instead of using an environment variable
- An assistant that answers from the LLM's own knowledge instead of the retrieved context — no real grounding
- Testing on only 1–2 questions before the live demo
- A notebook that only runs in the author's original cell order (Kernel → Restart & Run All should work)